Week 10 Lab: Machine Learning & Dashboards

INSY 8413 Introduction to Big Data Analytics, AUCA. Instructor: Prince Ishimwe

Name: Mucyo Gad Student ID: _____ Program: _____

Part 1 · Explore & Split

Setup: load the data

```
import pandas as pd
import numpy as np
import plotly.express as px
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_absolute_error, r2_score, accuracy_score, confusion_matrix

df = pd.read_csv("week10_students.csv")
df.head()
```

```
  student_id               name gender              program  district  \
0    AUCA001        Aline Uwase      F   Information Systems    Gasabo
1    AUCA002     Eric Iradukunda      M  Software Engineering   Musanze
2    AUCA003   Diane Uwizeyimana      F            Networking   Muhanga
3    AUCA004  Jean Bosco Ikirezi      M            Accounting  Kicukiro
4    AUCA005    Patrick Ingabire      M   Information Systems      Huye

   attendance_pct  quiz_avg  assignments_avg  score
0              55      31.0             35.0   28.6
1              73      61.1             76.7   64.5
2              92      35.2             65.5   52.7
3              69      65.2             54.2   68.3
4              88      39.3             42.9   42.3
```

Exercise 1.1: Know your data

```
print("Shape:", df.shape)
print()
print(df[["score", "quiz_avg", "assignments_avg", "attendance_pct"]].corr()["score"])
```

```
Shape: (150, 9)

score              1.000000
quiz_avg           0.800726
assignments_avg    0.535983
attendance_pct     0.238118
Name: score, dtype: float64
```

Exercise 1.2: Frame & split

```
features = ["quiz_avg", "assignments_avg", "attendance_pct"]
X = df[features]
y = df["score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

print("Train size:", len(X_train))   # 112
print("Test size :", len(X_test))    # 38
```

```
Train size: 112
Test size : 38
```

Part 2 · Regression: Predict the Score

Exercise 2.1: Train & read the model

```
reg = LinearRegression()
reg.fit(X_train, y_train)

for name, coef in zip(features, reg.coef_):
    print(f"{name:18s} coefficient = {coef:.3f}")
print(f"{'intercept':18s} = {reg.intercept_:.3f}")
```

```
quiz_avg           coefficient = 0.579
assignments_avg    coefficient = 0.365
attendance_pct     coefficient = 0.113
intercept          = -4.428
```

Exercise 2.2: Judge it honestly (test set)

```
y_pred = reg.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MAE = {mae:.2f} points")   # checkpoint about 3.3
print(f"R2  = {r2:.2f}")           # checkpoint about 0.89
```

```
MAE = 3.33 points
R2  = 0.89
```

Exercise 2.3: Predict a new student

```
new_student = pd.DataFrame({
    "quiz_avg": [70],
    "assignments_avg": [75],
    "attendance_pct": [90],
})
print("Predicted score (new student):", round(reg.predict(new_student)[0], 1))

me = pd.DataFrame({
    "quiz_avg": [65],
    "assignments_avg": [80],
    "attendance_pct": [95],
})
print("Predicted score (me):", round(reg.predict(me)[0], 1))
```

```
Predicted score (new student): 73.7
Predicted score (me): 73.2
```

Part 3 · Classification: Catch the At-Risk Students

Exercise 3.1: Build the target & train

```
df["passed"] = (df["score"] >= 50).astype(int)
print(df["passed"].value_counts())   # 1 = pass, 0 = fail

yc = df["passed"]
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    X, yc, test_size=0.25, random_state=42
)

clf = LogisticRegression(max_iter=1000)
clf.fit(Xc_train, yc_train)
```

```
passed
1    131
0     19
Name: count, dtype: int64

LogisticRegression(max_iter=1000)
```

Exercise 3.2: The accuracy trap

```
yc_pred = clf.predict(Xc_test)

acc = accuracy_score(yc_test, yc_pred)
baseline = yc_test.mean()   # lazy model: predict "pass" for everyone

print(f"Model accuracy        : {acc:.3f}")
print(f"Lazy baseline accuracy: {baseline:.3f}")
print("Confusion matrix (rows = actual, cols = predicted):")
print(confusion_matrix(yc_test, yc_pred))
```

```
Model accuracy        : 0.895
Lazy baseline accuracy: 0.895
Confusion matrix (rows = actual, cols = predicted):
[[ 3  1]
 [ 3 31]]
```

Part 4 · Feature Engineering

Exercise 4.1: Does more information help?

```
def test_r2(X_in):
    Xtr, Xte, ytr, yte = train_test_split(X_in, y, test_size=0.25, random_state=42)
    m = LinearRegression().fit(Xtr, ytr)
    return r2_score(yte, m.predict(Xte))

X_v1 = df[["quiz_avg"]]
X_v2 = df[features]
X_v3 = pd.get_dummies(df[features + ["program"]], columns=["program"], drop_first=True)

results = pd.DataFrame({
    "version": ["1. quiz_avg only", "2. three numeric features", "3. three + program (one-hot)"],
    "test_R2": [test_r2(X_v1), test_r2(X_v2), test_r2(X_v3)],
})
print(results.round(3).to_string(index=False))   # checkpoint about 0.62 -> 0.89 -> 0.88
```

```
                   version  test_R2
           1. quiz_avg only    0.624
   2. three numeric features    0.887
3. three + program (one-hot)    0.880
```

Part 5 · Ship a Dashboard

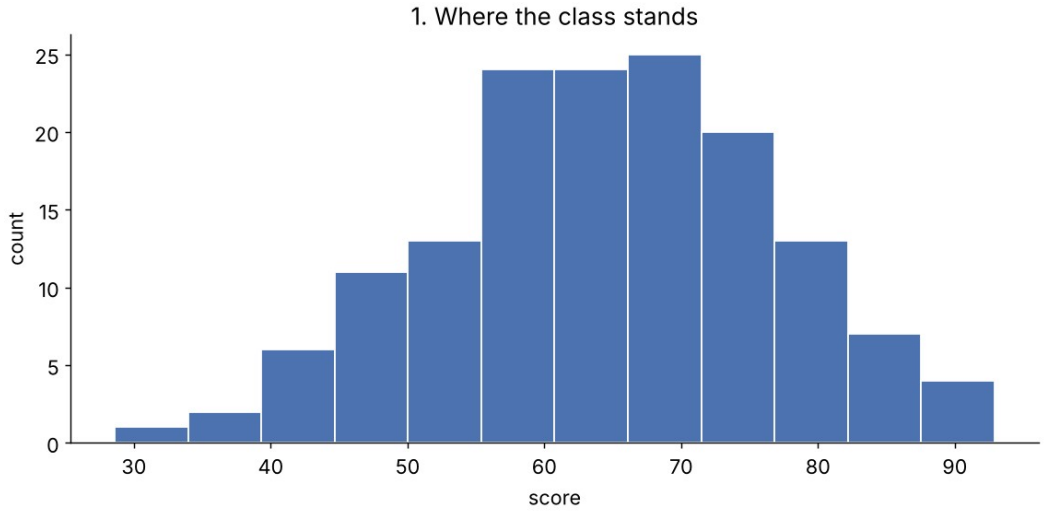
Exercise 5.1: The story in three charts

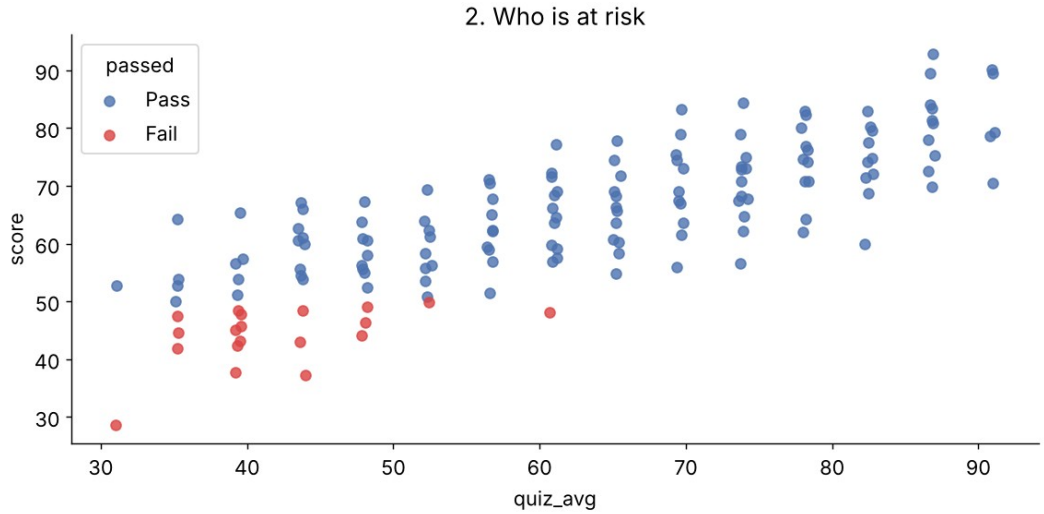
```
df_plot = df.copy()
df_plot["passed"] = df_plot["passed"].map({1: "Pass", 0: "Fail"})

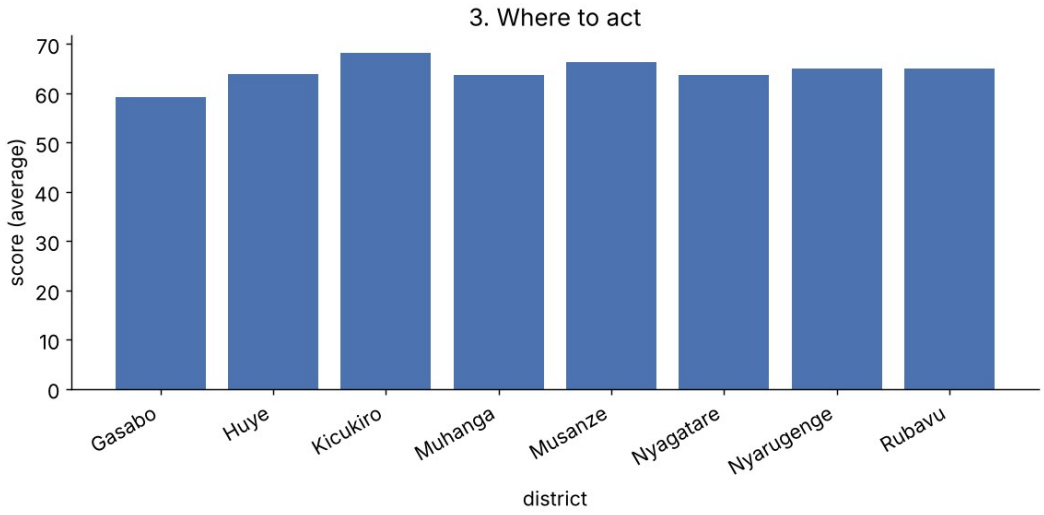
fig1 = px.histogram(df_plot, x="score", nbins=12, title="1. Where the class stands")
fig1.show()

fig2 = px.scatter(df_plot, x="quiz_avg", y="score", color="passed",
                  hover_name="name", title="2. Who is at risk")
fig2.show()

district_avg = df_plot.groupby("district", as_index=False)["score"].mean()
fig3 = px.bar(district_avg, x="district", y="score", title="3. Where to act")
fig3.show()
```



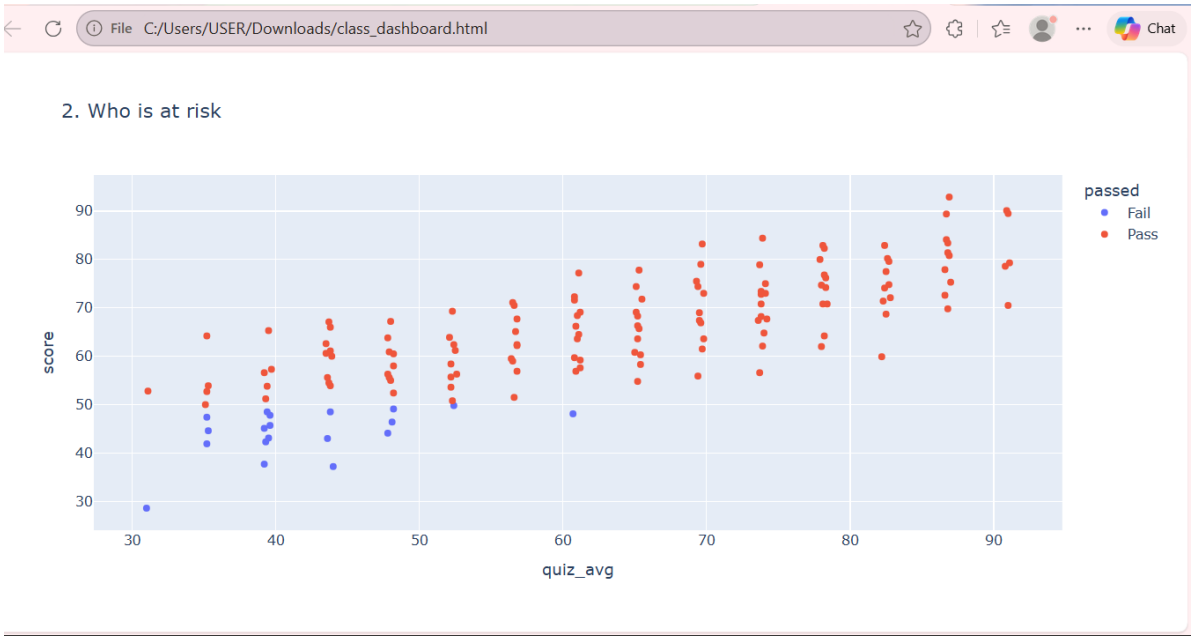




Exercise 5.2: Publish it

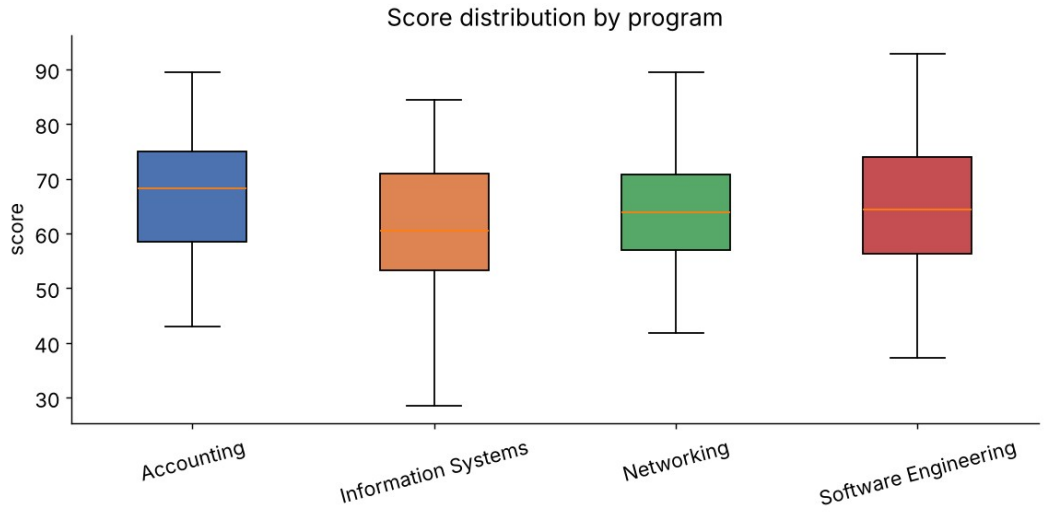
```
fig2.write_html("class_dashboard.html")

from google.colab import files
files.download("class_dashboard.html")
```



Bonus: One chart, four stories

```
fig4 = px.box(df, x="program", y="score", color="program",
              title="Score distribution by program")
fig4.show()
```




```
print(df.groupby("program")["score"].agg(["std", "min", "max"]).round(1))
```

```
                      std   min   max
program
Accounting            11.0  43.0  89.5
Information Systems   12.3  28.6  84.4
Networking            10.8  41.9  89.4
Software Engineering  13.8  37.2  92.9
```